



Right now when I try to jump on the platforms I either fall right through, or end up standing on mid air. The platforms actual location and where the game seems to think they are, are different. Please make it so when the character jumps onto a platform it works correctly, and they actually stand on a platform not mid air.

Diagnosed character-platform collision offset mismatch causing visual misalignment

Found it — this is a real, provable bug, not just a tuning issue.

The cause: `drawChar()` draws the sprite offset by `CHAR_DRAW_OFFSET = -164`, so the character's actual *visual* feet position on screen is `charY + CHAR_DRAW_OFFSET`, not `charY` itself. Your ground collision (`charY = gy`) still looks correct only because `GROUND_TOP_CANVAS_Y` was hand-tuned by eye against that same offset. But the new platform landing code sets `charY = topY` directly — with **no such compensation** — so whenever the character "lands" on a rock, the sprite is drawn exactly 164 screen-pixels *above* where the collision thinks the feet are. That's a constant, guaranteed gap (it survives zoom and camPanY, since both cancel out of the math identically for platform and character — I checked). It explains both symptoms: the character visibly floats above rocks it's "standing" on, and because the actual (invisible) collision box is 164px below the visible sprite, players walk right off it thinking they're still on solid rock — reads as "falling through."

The fix: make platform landings solve for `charY` in terms of the *visual* feet position, the same way ground already (implicitly) does — instead of `charY = topY`, use `charY = topY - CHAR_DRAW_OFFSET`, and check proximity/crossing using `charY + CHAR_DRAW_OFFSET` (the true visual feet) rather than raw `charY`. Ground code is untouched since it already works.

Replace the **Pass 1** block:

```
javascript
```

```

if (standingPlatformIndex !== -1) {
  let g = PLATFORM_GEOM[standingPlatformIndex];
  let scx = climbScreenX(g.cx);
  let scy = climbScreenY(g.cy);
  let dw = g.w * s * PLATFORM_DRAW_SCALE;
  let dh = g.h * s * PLATFORM_DRAW_SCALE;
  let sx0 = scx - dw / 2;
  let sx1 = scx + dw / 2;
  let topY = (scy - dh / 2) + dh * 0.12;
  let marginX = 10 * s;
  let withinX = charX > sx0 + marginX && charX < sx1 - marginX;
  // Compare the character's VISIBLE feet (charY + CHAR_DRAW_OFFSET),
  // not raw charY — otherwise the sprite renders CHAR_DRAW_OFFSET
  // pixels away from the surface it's supposedly standing on.
  let visFeet = charY + CHAR_DRAW_OFFSET;
  let nearSurface = visFeet <= topY + 8 && visFeet >= topY - 8;
  if (withinX && nearSurface && vely >= 0) {
    // Solve for the charY that puts the VISUAL feet exactly at topY.
    charY = topY - CHAR_DRAW_OFFSET; vely = 0; onGround = true;
  } else {
    standingPlatformIndex = -1; // walked off the edge or jumped away
  }
}

```

Replace the **Pass 2** crossing check:

javascript

```

let topY = (scy - dh / 2) + dh * 0.12; // small inset from the rock's top sil
let marginX = 10 * s;
let withinX = charX > sx0 + marginX && charX < sx1 - marginX;
// Same visual-feet correction as Pass 1 — compare where the
// sprite will actually be drawn, not the raw physics charY.
let prevFeet = prevY + CHAR_DRAW_OFFSET;
let currFeet = charY + CHAR_DRAW_OFFSET;
if (withinX && prevFeet <= topY && currFeet >= topY && vely > 0) {
  charY = topY - CHAR_DRAW_OFFSET; vely = 0; onGround = true;
  standingPlatformIndex = i;
  break;
}

```

Nothing else needs to change — `drawDebugPlatformBoxes()` still shows the true rock surface correctly, and ground collision (`gy`) is left exactly as-is since it was never broken. After this, press `P` in-game to toggle the debug overlay and confirm the character's feet now sit right on the red line instead of floating above it.

Can you also make it so the platforms don't go below where the ground is? The lowest platform can't be lower than the ground level. You can move around the platforms if you need.

Devised clamping logic to anchor platforms above ground level

Good catch — with `SPREAD_FACTOR_Y = 2.0` some of the lower platforms in `PLATFORM_BOXES` can end up pushed down past `GROUND_TOP_CANVAS_Y`, i.e. literally below/inside the ground sprite.

The cleanest fix: clamp each platform's **rendered bottom edge** (not just its raw box) to sit at or above the ground line, in the same canvas-space units `GROUND_TOP_CANVAS_Y` is defined in. I use the *rendered* half-height (`* PLATFORM_DRAW_SCALE`) so the clamp matches what's actually drawn on screen, not the smaller pre-scale box — otherwise a platform could still visually dip into the ground even though its raw box looked fine.

Add this right after `computeSpreadBoxes()` is defined, and change how `PLATFORM_BOXES` is built:

```
javascript
```

```

// Never let a platform's RENDERED bottom edge sink below the ground
// line — a platform overlapping/under the ground looks broken, and
// its landing surface would be unreachable anyway. Clamped in the
// same canvas-space units as GROUND_TOP_CANVAS_Y, using each
// platform's rendered half-height (src height * PLATFORM_DRAW_SCALE)
// so the clamp matches what's actually drawn, not the smaller
// pre-scale box.
const PLATFORM_GROUND_MARGIN = 20; // canvas px of breathing room above the ground
function clampAboveGround(boxes, srcBoxes) {
  return boxes.map((b, i) => {
    let srcH = srcBoxes[i][3] - srcBoxes[i][1];
    let halfH = (srcH * PLATFORM_DRAW_SCALE) / 2;
    let h = b[3] - b[1];
    let cy = (b[1] + b[3]) / 2;
    let maxCy = GROUND_TOP_CANVAS_Y - halfH - PLATFORM_GROUND_MARGIN;
    if (cy > maxCy) cy = maxCy;
    return [b[0], cy - h / 2, b[2], cy + h / 2];
  });
}

// The boxes actually used for physics + on-screen placement.
const PLATFORM_BOXES = clampAboveGround(
  computeSpreadBoxes(PLATFORM_SRC_BOXES, SPREAD_FACTOR_X, SPREAD_FACTOR_Y),
  PLATFORM_SRC_BOXES
);

```

This replaces the existing line:

```

javascript

const PLATFORM_BOXES = computeSpreadBoxes(PLATFORM_SRC_BOXES, SPREAD_FACTOR_X, SPRE

```

A few notes on why this approach and not alternatives:

- I clamp *per-platform*, not by shifting the whole group up uniformly — shifting everything up to fix the lowest platform would waste your carefully-tuned vertical spread on the platforms that were already fine.
- `PLATFORM_DRAW_SCALE` is already declared earlier in the file, so this drops in without reordering anything.
- Nothing downstream (`PLATFORM_GEOM`, collision, `drawClimbScene()`) needs to change — they already just consume `PLATFORM_BOXES`.

If you want the platforms to *keep* their full vertical spread and instead have the lowest ones squeeze closer together near the ground rather than all bottoming out at the same clamp line, let me know — that'd need a slightly different (per-platform proportional  compression) approach instead of a hard clamp.